

WG · APD

dependency injection

ON DI, BEST PRACTICES, MITIGATIONS

Presented by Dzaki and Nicholas

Contents

-
- 01 Pre-DI: The Problem
 - 02 What Is DI? · The Car Analogy · How Is DI Performed?
 - 03 Why DI Is Good For Applications
 - 04 When DI Should / Should Not Be Used
 - 05 How Does DI Look Like In Code — Frameworks
 - 06 DI Anti-Patterns
 - 07 Cons Of DI
 - 08 Pros Of DI (Cons Mitigation)
 - 09 Quiz Time
-

PROBLEM

Pre-DI: The Problem

WHY TIGHTLY-COUPLED CODE BECOMES BRITTLE AND HARD TO TEST

1 Tightly-Coupled Code

ANTI-PATTERN

```
class OrderService {  
    private StripeGateway payment = new StripeGateway(); // hardcoded!  
    private FileLogger logger = new FileLogger(); // hardcoded!  
  
    void placeOrder(Order order) {  
        logger.log("Placing order " + order.id);  
        payment.charge(order.total);  
    }  
}
```

WHY THIS HURTS

1. Hardcoded.

OrderService is welded to StripeGateway and FileLogger by name — switching to PayPal means editing OrderService itself.

2. Untestable.

A unit test for placeOrder() always hits the real Stripe API and writes a real log file. There's no way to substitute a fake.

3. Backwards dependency.

The high-level OrderService depends directly on low-level concrete classes, not on abstractions.

4. Duplicated setup.

Every class that needs a payment gateway repeats the same new StripeGateway() line.

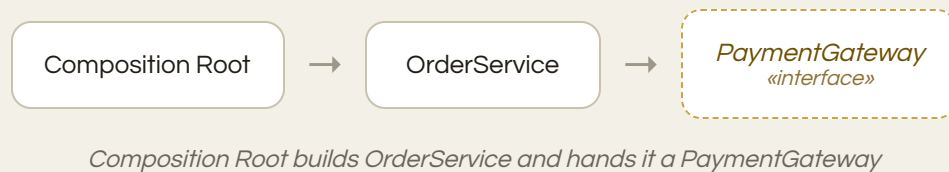
THE SETUP FOR WHAT'S NEXT

This is exactly the problem DI solves — instead of a class creating its own dependencies, something else hands them in from outside.

DEFINITION

What Is Dependency Injection?

THE CORE IDEA, AND HOW IT RELATES TO INVERSION OF CONTROL



1. Depend on abstractions.

OrderService should hold a reference to a PaymentGateway interface, not a concrete StripeGateway.

2. Receive, don't construct.

Instead of `new StripeGateway()` inside OrderService, the concrete implementation is handed in from outside — typically as a constructor parameter.

3. Someone else decides.

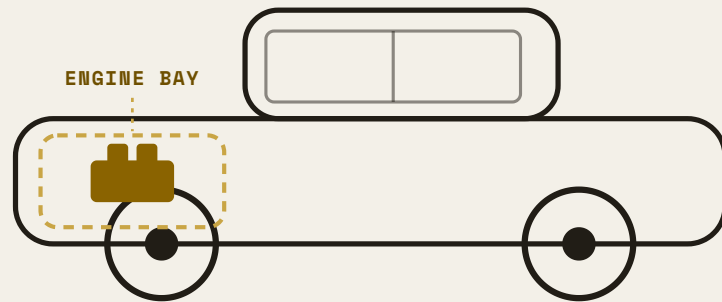
A separate piece of code — a "composition root," or a DI framework/container — decides which concrete implementation to hand over.

DI VS. IOC — A COMMON MIX-UP

DI is one way to implement a bigger idea called **Inversion of Control**. Control over *which* implementation gets used is inverted — it moves out of the class itself and into whoever wires the app together.

Same Car, Different Engine

A PICTURE OF DI THAT DOESN'T NEED A SINGLE LINE OF CODE



OrderService

The chassis stays identical. Only the engine plugged into its one socket changes.

PLUG IN AN ENGINE

● Gas Engine
StripeGateway

● Electric Motor
PayPalGateway

● Test Rig
FakePaymentGateway

Car chassis	=	OrderService
Engine bay	=	PaymentGateway interface
Swappable engine	=	Concrete implementation

THE TAKEAWAY

The **car body** never changes — it only expects something that fits the **engine bay**. Swapping Gas → Electric → Test Rig is exactly what swapping StripeGateway → PayPalGateway → a fake looks like in DI: the dependency changes, the dependent class doesn't.

How DI Is Performed

3 WAYS TO INJECT A DEPENDENCY

1

Constructor Injection

DEFAULT CHOICE

```
class OrderService {
    private final PaymentGateway payment;

    OrderService(PaymentGateway payment) {
        this.payment = payment;
    }
}
```

Dependencies arrive as constructor parameters. The object can never exist in a half-configured state — if it exists, its dependencies are set.

2

Setter Injection

FOR OPTIONAL DEPS

```
class OrderService {
    private PaymentGateway payment;

    void setPaymentGateway(PaymentGateway p) {
        this.payment = p;
    }
}
```

Dependencies are set via a public setter after construction. Useful for optional dependencies, but the object can briefly exist unconfigured.

3

Interface Injection

RARELY USED TODAY

```
interface PaymentAware {
    void injectPayment(
        PaymentGateway p);
}

class OrderService
    implements PaymentAware {
    public void injectPayment(
        PaymentGateway p) {
        this.payment = p;
    }
}
```

The class implements an interface exposing an inject method, which a framework calls. More ceremony, so it has mostly fallen out of favor.

Which one should you use? Constructor injection is the default in modern frameworks — it makes required dependencies explicit and impossible to forget. Reach for setter injection only when a dependency is genuinely optional.

BENEFITS

Why DI Is Good For Applications

THE PAYOFF FOR THE EXTRA INDIRECTION

1

Testability

Swap StripeGateway for a FakePaymentGateway in tests. No real API calls, no real charges, fast and deterministic tests of placeOrder() alone.

2

Loose Coupling

OrderService only knows about the PaymentGateway interface, never a concrete class — the two can change independently of each other.

3

Flexibility & Swappability

Switching from Stripe to PayPal is a one-line change where OrderService is wired up, not a rewrite of OrderService itself.

4

Follows the Dependency Inversion Principle

The "D" in SOLID: both high-level and low-level modules depend on abstractions, not on each other directly.

5

Centralized Configuration

Every "which implementation goes where" decision lives in one place — a composition root or container config — not scattered through the codebase.

TRADE-OFFS

When To (Not) Use DI

IT'S NOT FREE — RECOGNIZE WHEN IT EARNS ITS KEEP

REACH FOR DI WHEN...

- ✓ You have multiple interchangeable implementations — different payment gateways, storage backends, or environments (dev/test/prod).
- ✓ You need to unit test business logic in isolation from external systems (databases, APIs, the file system).
- ✓ The app is large enough that wiring dependencies by hand everywhere becomes repetitive and error-prone.
- ✓ You're following a clean/hexagonal architecture and want business logic decoupled from infrastructure.

THINK TWICE WHEN...

- ✗ The class has one, permanent, simple dependency that will never change or need mocking (e.g. a Math utility).
- ✗ It's a small script or prototype — a full DI framework adds setup overhead with no real payoff yet.
- ✗ The extra indirection makes a small codebase harder to read and trace than it needs to be.
- ✗ Performance is extremely critical and a reflection-based DI framework adds unacceptable overhead (rare, but real).

EXAMPLE

DI In Code

FROM MANUAL WIRING TO A REAL FRAMEWORK

Manual DI — no framework

```
// composition root — one place that wires the whole app
PaymentGateway payment = new StripeGateway(apiKey);
Logger logger = new FileLogger("app.log");
OrderService orderService =
    new OrderService(payment, logger);
```

Nothing magic here — just one place ("the composition root") that constructs everything and passes it along. This is DI even without a framework.

Spring (Java) — annotation-driven

```
@Component
class StripeGateway implements PaymentGateway { /* ... */ }

@Component
class OrderService {
    private final PaymentGateway payment;

    @Autowired
    OrderService(PaymentGateway payment) { this.payment = payment; }
}
```

The framework scans for `@Component` classes, works out the dependency graph, and wires it for you. Same idea, automated.

When DI Becomes the Problem

INJECTION EXPOSES DESIGN PROBLEMS – IT DOES NOT AUTOMATICALLY SOLVE THEM

```
class CheckoutService {  
    CheckoutService(  
        CartRepository carts,  
        PricingService pricing,  
        InventoryService inventory,  
        PaymentGateway payments,  
        FraudChecker fraud,  
        ShippingService shipping,  
        EmailSender email,  
        Analytics analytics  
    ) { /* DI did not make this cohesive */ }  
}
```

The constructor is giving useful feedback: this class probably owns too many responsibilities. Hiding those dependencies behind a container would only hide the smell.

Injecting everything

Values and tiny pure helpers do not need an interface, factory, and registration “just in case.” Add seams where change or testing needs them.

Constructor over-injection

A long parameter list is usually a cohesion problem. Split the class around capabilities or use-cases instead of bundling dependencies.

Service locator

Calling `container.get()` inside business code hides required dependencies and moves failures to runtime.

One interface per class

An interface with one permanent implementation can be ceremony without a boundary. Abstract behavior, not every concrete type.

The real test

DI should make boundaries clearer. If the wiring grows faster than the behavior, revisit the design—not merely the container configuration.

Cons Of DI

THE HONEST TRADE-OFFS

1

Adds Indirection

"Jump to definition" no longer shows what actually runs — you have to know how the wiring works ("magic").

2

More boilerplate for small projects

Interfaces, constructors, and configuration add ceremony that a five-file script simply doesn't need.

3

A real learning curve

Heavyweight frameworks like Spring can feel opaque until you understand what the container is doing.

4

Errors surface at runtime

A missing or misconfigured dependency often only fails when the app starts, not at compile time.

5

Can be over-used

Injecting everything "just in case" adds abstraction and files without any real payoff.

6

Possible performance overhead

Reflection-heavy containers add startup and sometimes runtime cost — rarely decisive, but real.

STRENGTHS

Pros Of DI

WHY TEAMS REACH FOR IT

1

Dramatically Improves Testability

Mock or stub any dependency — tests run fast, deterministic, and never touch a real database or API.

2

Promotes Loose Coupling

High-level code depends on interfaces, not concrete classes, so either side can change independently.

3

Makes Swapping Implementations Trivial

Changing a payment provider or a storage backend becomes a wiring change, not a rewrite.

4

Encourages Coding To Interfaces

Because a dependency must be injected as an abstraction, the design ends up more interface-driven overall.

5

Centralizes Construction Logic

Object graphs are built in one predictable place instead of scattered new calls across the codebase.

6

Supports Flexible Object Lifetimes

Containers can manage singleton, per-request, or transient lifetimes for you, consistently.

WRAP-UP

qna time

LET'S SEE WHAT STUCK

Fire away — Dzaki and Nicholas